VLSI Design Internship – Task 3

Verilog RTL Design of Sequential Circuits and Flip-Flops

Part A: Introduction to Sequential Logic

Objective

In Task-2, the fundamentals of Verilog HDL, RTL design methodology, and combinational logic circuits (logic gates, half adder, and full adder) were covered. Task-3 builds on this by moving from combinational circuits to sequential circuits, which form the basis of processors, controllers, and memory systems. Unlike combinational logic, sequential circuits depend on both the current inputs and the previous state, and require memory elements that operate synchronously with a clock signal.

The objective of this task was to understand sequential logic behavior, implement flip-flops using Verilog, design simple registers and counters, write testbenches for sequential circuits, and simulate and analyze clock-driven circuit behavior.

Circuits Implemented

- D Flip-Flop
- JK Flip-Flop
- 4-bit Register
- 4-bit Binary Counter

Tool Used

EDA Playground (online Verilog/SystemVerilog IDE) with Verilator v5.044 as the simulation engine, and EPWave for waveform visualization.

Part B: Design and Verification of D Flip-Flop

Aim

To design, simulate, and verify the operation of a positive edge-triggered D Flip-Flop using Verilog HDL in EDA Playground.

Theory

A D (Data) Flip-Flop is a sequential logic circuit that stores one bit of data. Unlike combinational circuits, its output depends on both the current input and the previous state. It changes its output only on the positive (rising) edge of the clock signal. The D Flip-Flop is widely used in registers, shift registers, counters, memory circuits, and digital systems.

Working Principle

- When Reset (rst) = 1, the output becomes 0.
- When Reset (rst) = 0, the output copies the input (D) only at the rising edge of the clock.
- Between clock edges, the output remains unchanged.

Truth Table

Clock Edge	Reset	D	Q (Next State)
↑	1	X	0
↑	0	0	0
↑	0	1	1

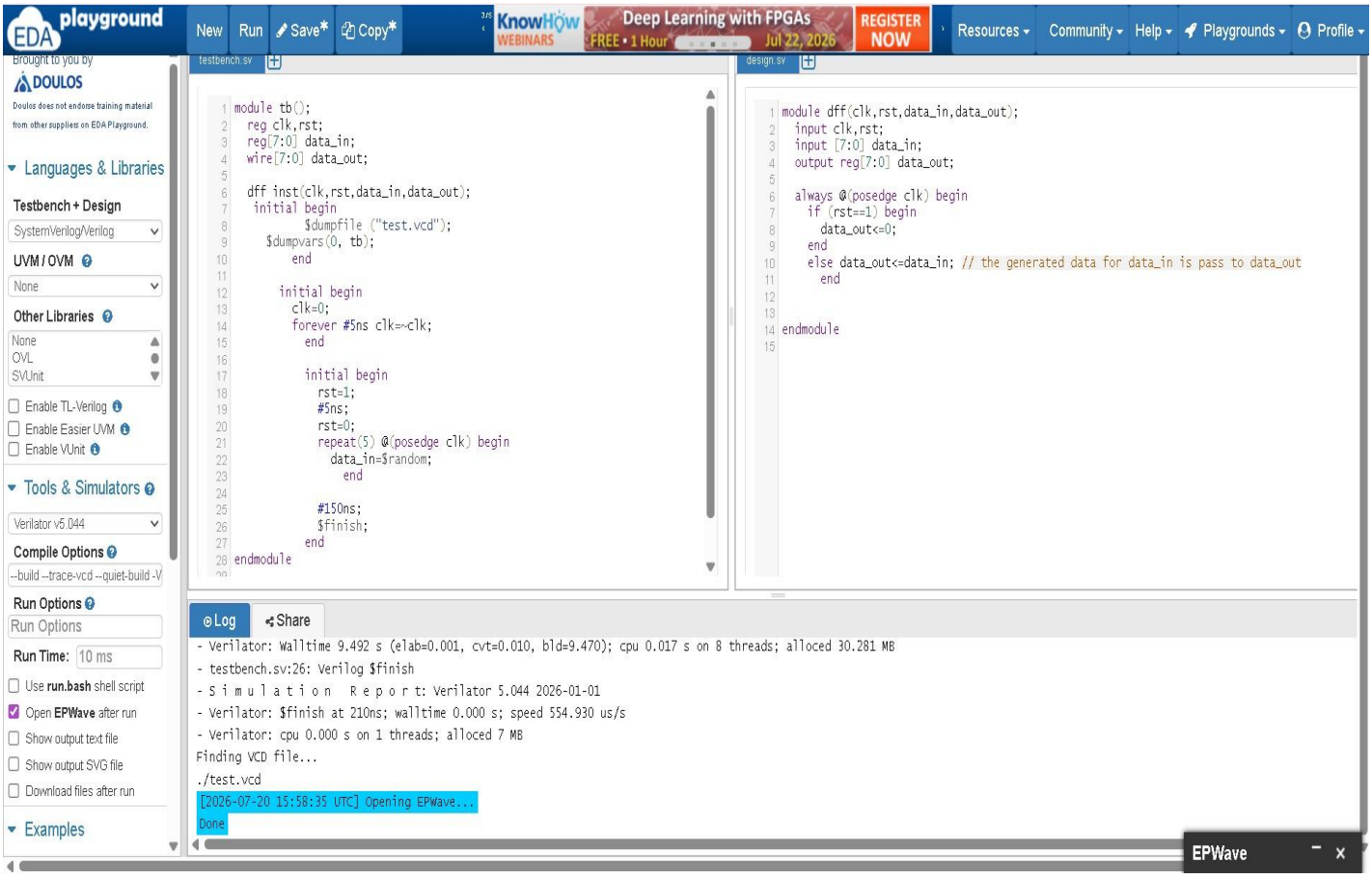
Verilog Design Code (as implemented in EDA Playground)

```
module dff(clk, rst, data_in, data_out);
  input clk, rst;
  input [7:0] data_in;
  output reg [7:0] data_out;

  always @(posedge clk) begin
    if (rst == 1) begin
      data_out <= 0;
    end
    else data_out <= data_in;  // the generated data for data_in is pass to
data_out
  end

endmodule
```

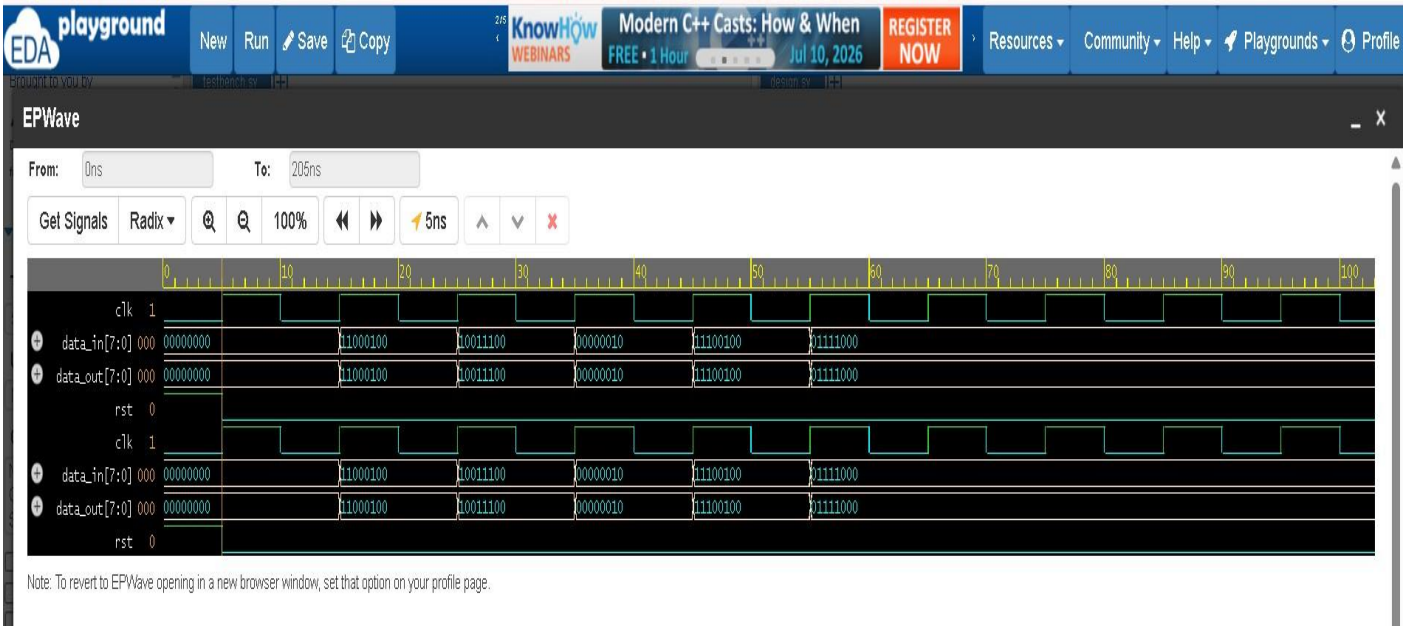
Testbench Code

```
module tb();
  reg clk, rst;
  reg [7:0] data_in;
  wire [7:0] data_out;

  dff inst(clk, rst, data_in, data_out);
  initial begin
    $dumpfile ("test.vcd");
    $dumpvars(0, tb);
  end

  initial begin
    clk = 0;
    forever #5ns clk = ~clk;
  end

  initial begin
    rst = 1;
    #5ns;
    rst = 0;
    repeat(5) @(posedge clk) begin
      data_in = $random;
    end
    #150ns;
    $finish;
  end
endmodule
```

Screenshot 1: D Flip-Flop – Verilog Design and Testbench Code (EDA Playground)



Screenshot 2: D Flip-Flop – Simulation Waveform (EPWave)



Simulation Procedure

- Write the Verilog code for the D Flip-Flop.
- Create a testbench.
- Generate a clock signal.
- Apply reset.
- Apply random input values.
- Simulate the design.
- Observe the waveform using EPWave.

Waveform Observation

The waveform obtained during simulation shows the following: initially, Reset = 1, therefore data_out = 00000000. After reset becomes 0, different random values are applied to data_in. At every positive edge of the clock, data_out captures the value of data_in. Between clock edges, data_out remains constant. Observed input values include 11000100, 10011100, 00000010, 11100100, and 01111000. The output follows these values exactly at the rising edge of the clock.

Output Verification

Clock Rising Edge	Reset	data_in	data_out	Status
5 ns	1	XXXXXXXX	00000000	Pass
15 ns	0	11000100	11000100	Pass
25 ns	0	10011100	10011100	Pass
35 ns	0	00000010	00000010	Pass
45 ns	0	11100100	11100100	Pass
55 ns	0	01111000	01111000	Pass

Result

The positive edge-triggered D Flip-Flop was successfully designed, simulated, and verified using Verilog HDL. The simulation waveform confirms that the output updates only at the rising edge of the clock and correctly follows the input after the reset is deactivated. Hence, the design functions as expected.

Part C: Designing a JK Flip-Flop

Aim

To design and simulate a JK Flip-Flop using Verilog HDL in EDA Playground and verify its operation using the EPWave waveform.

Theory

A JK Flip-Flop is a sequential logic circuit that stores one bit of data. It is an improved version of the SR Flip-Flop because it eliminates the invalid input condition. The output changes only on the positive (rising) edge of the clock. The JK Flip-Flop has two inputs, J (Set input) and K (Reset input), and one output, Q. Depending on the values of J and K at the rising edge of the clock, the output changes according to the truth table.

Truth Table

Clock Edge	J	K	Q (next)
↑	0	0	No Change
↑	0	1	Reset (Q = 0)
↑	1	0	Set (Q = 1)
↑	1	1	Toggle ($Q = \overline{Q}$)

Working

- J = 0, K = 0: The output retains its previous value.
- J = 0, K = 1: The output is reset to logic 0.
- J = 1, K = 0: The output is set to logic 1.
- J = 1, K = 1: The output toggles its previous state on every positive clock edge.

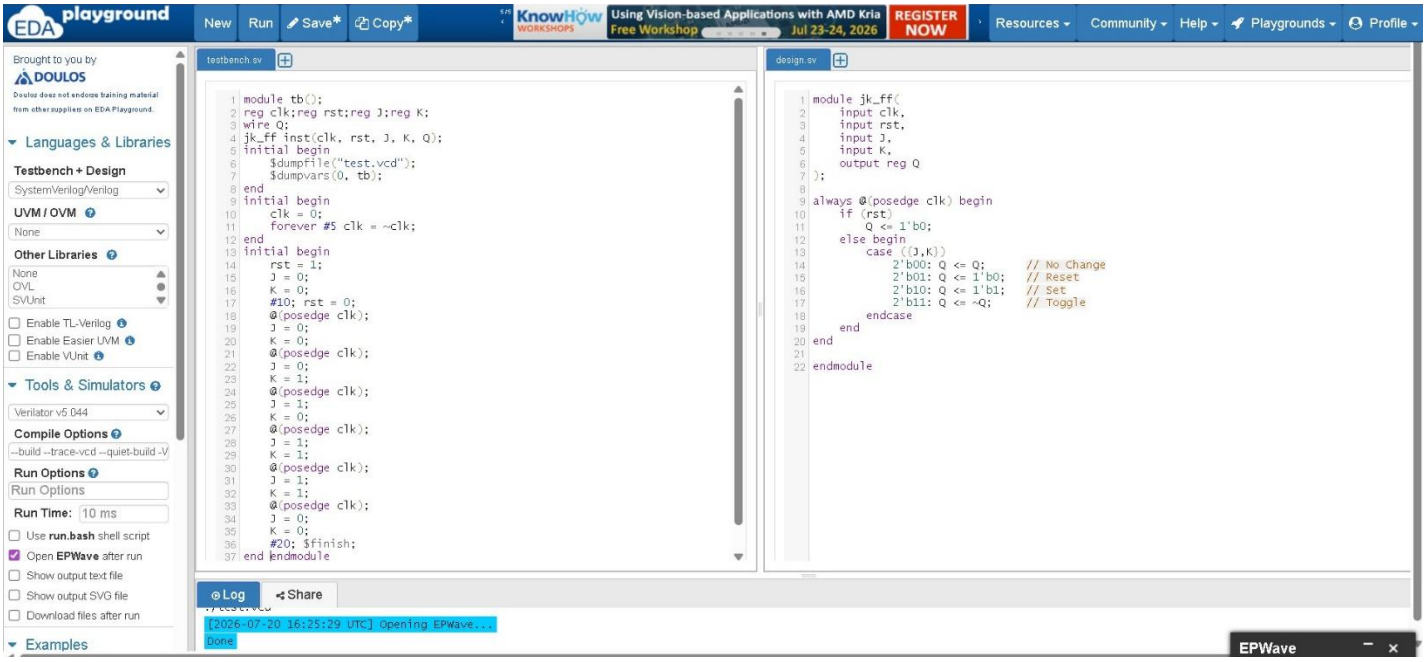
Verilog Design Code (as implemented in EDA Playground)

```
module jk_ff(
    input clk,
    input rst,
    input J,
    input K,
    output reg Q
);

  always @(posedge clk) begin
    if (rst)
      Q <= 1'b0;
    else begin
      case ({J,K})
        2'b00: Q <= Q;      // No Change
        2'b01: Q <= 1'b0;   // Reset
        2'b10: Q <= 1'b1;   // Set
        2'b11: Q <= ~Q;     // Toggle
      endcase
    end
  end
endmodule
```

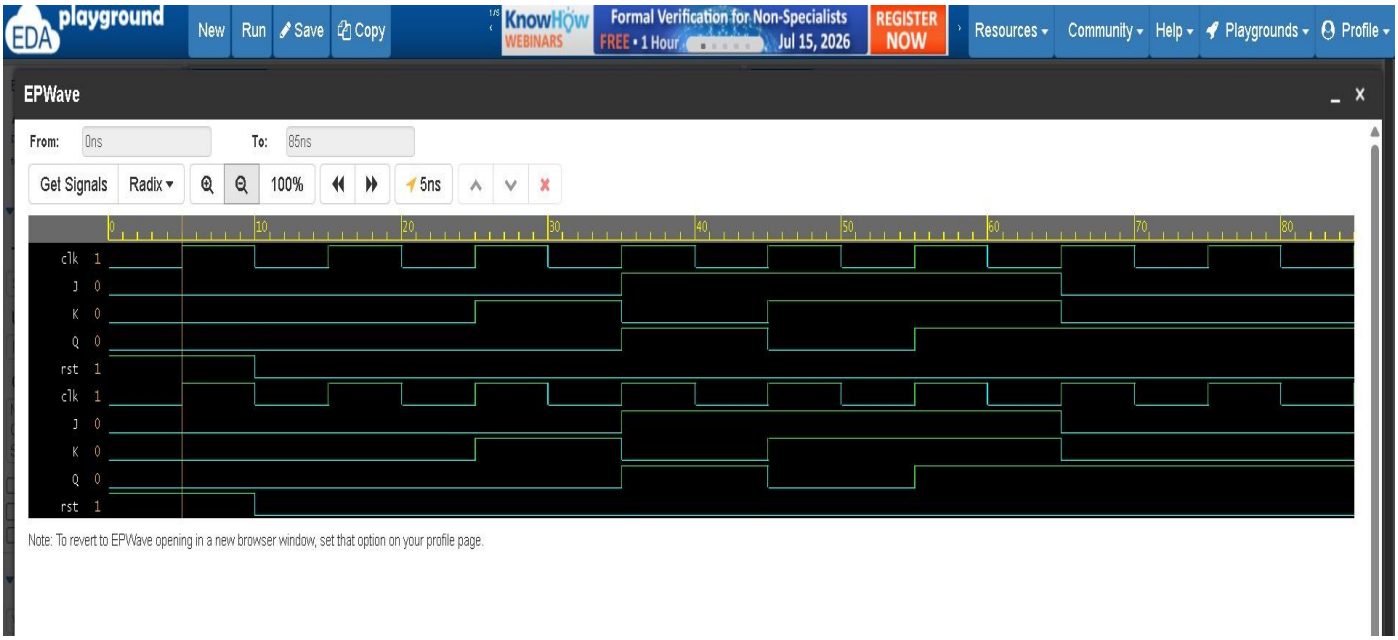
Testbench Code

```
module tb();
  reg clk; reg rst; reg J; reg K;
  wire Q;
  jk_ff inst(clk, rst, J, K, Q);
  initial begin
    $dumpfile("test.vcd");
    $dumpvars(0, tb);
  end
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  initial begin
    rst = 1;
    J = 0;
    K = 0;
    #10; rst = 0;
    @(posedge clk);
    J = 0; K = 0;
    @(posedge clk);
    J = 0; K = 1;
    @(posedge clk);
    J = 1; K = 0;
    @(posedge clk);
    J = 1; K = 1;
    @(posedge clk);
    J = 0; K = 0;
    #20; $finish;
  end
endmodule
```

Screenshot 1: JK Flip-Flop – Verilog Design and Testbench Code (EDA Playground)



Screenshot 2: JK Flip-Flop – Simulation Waveform (EPWave)



Simulation Output

The waveform obtained in EPWave verifies all the operations of the JK Flip-Flop. Initially, the reset signal initializes the output Q = 0. For J = 0 and K = 0, the output remains unchanged. For J = 0 and K = 1, the output is reset to 0. For J = 1 and K = 0, the output is set to 1. For J = 1 and K = 1, the output toggles between 0 and 1 on every positive edge of the clock. The simulation results match the expected truth table, confirming the correct operation of the JK Flip-Flop.

Observed Output

Time	J	K	Expected Output (Q)	Observation
Initial	X	X	0 (Reset)	Verified
0–20 ns	0	0	No Change	Verified
20–40 ns	0	1	Reset	Verified
40–60 ns	1	0	Set	Verified
60–80 ns	1	1	Toggle	Verified

Result

The JK Flip-Flop was successfully designed and simulated using Verilog HDL in EDA Playground. The waveform confirms that the circuit correctly performs No Change, Reset, Set, and Toggle operations according to the JK Flip-Flop truth table. Hence, the design is verified successfully.

Part D: Designing a 4-bit Register

Aim

To design and simulate a 4-bit Register using Verilog HDL in EDA Playground and verify its operation using the EPWave waveform.

Theory

A 4-bit Register is a sequential logic circuit used to store four bits of binary data. It is made up of four D Flip-Flops connected together, where each flip-flop stores one bit. The register stores the input data only on the positive (rising) edge of the clock signal. A reset signal is used to clear the register and initialize the output to 0000.

Inputs and Output

- clk – Clock signal
- rst – Reset signal
- D[3:0] – 4-bit input data
- Q[3:0] – 4-bit stored output

Working

- When rst = 1, the register is reset and the output becomes 0000.
- When rst = 0, the register stores the input data D at every positive edge of the clock.
- The output remains unchanged until the next rising edge of the clock.
- The register can store one 4-bit binary number at a time.

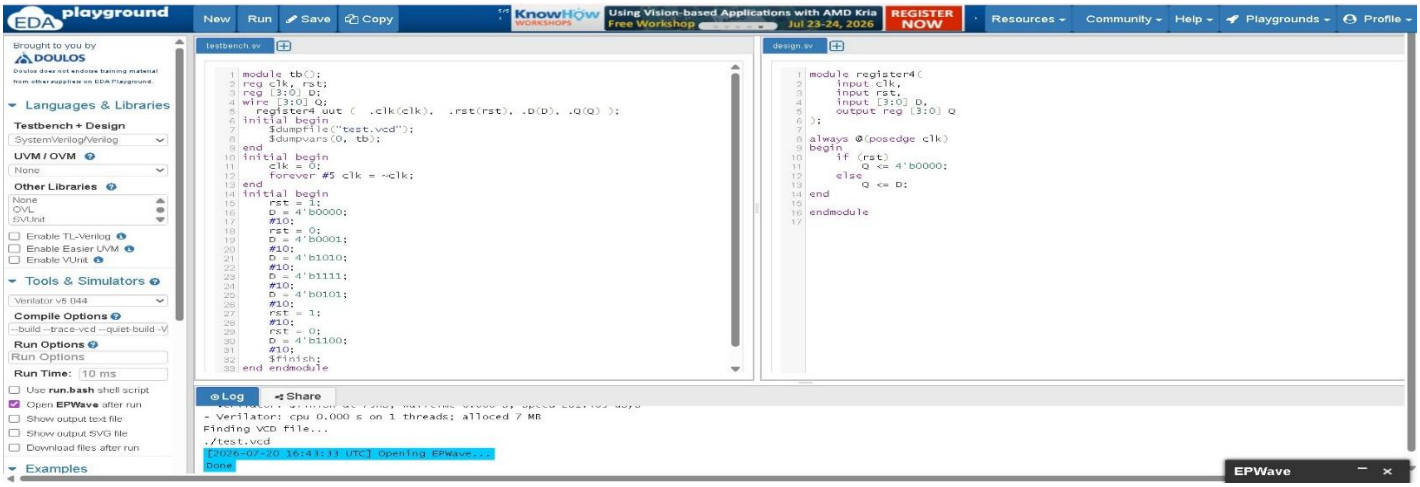
Verilog Design Code (as implemented in EDA Playground)

```
module register4(
    input clk,
    input rst,
    input [3:0] D,
    output reg [3:0] Q
);

  always @(posedge clk)
  begin
    if (rst)
      Q <= 4'b0000;
    else
      Q <= D;
  end

endmodule
```

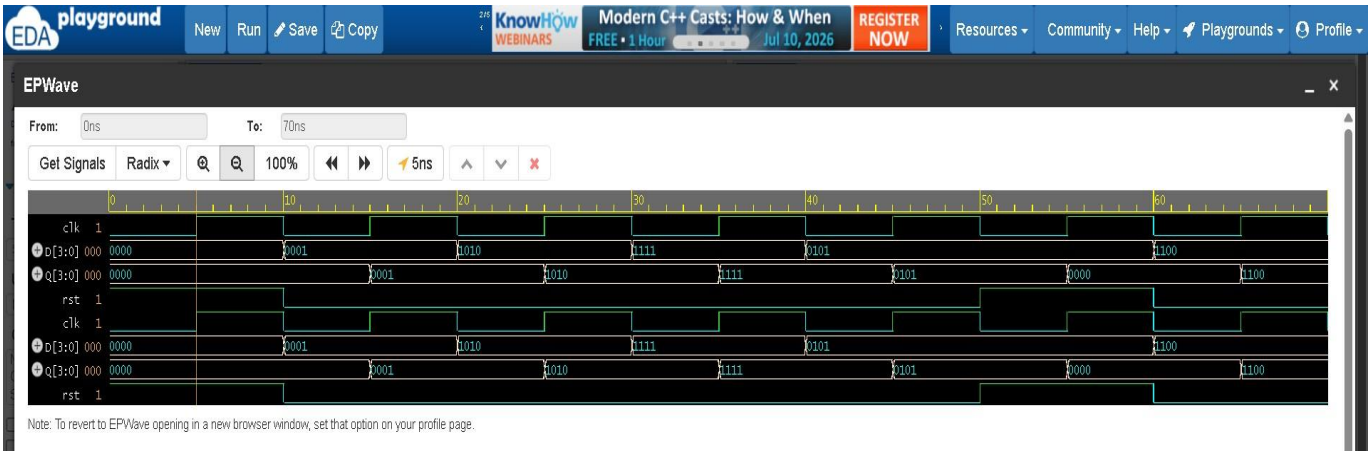
Testbench Code

```
module tb();
  reg clk, rst;
  reg [3:0] D;
  wire [3:0] Q;
  register4 uut ( .clk(clk), .rst(rst), .D(D), .Q(Q) );
  initial begin
    $dumpfile("test.vcd");
    $dumpvars(0, tb);
  end
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  initial begin
    rst = 1;
    D = 4'b0000;
    #10;
    rst = 0;
    D = 4'b0001;
    #10;
    D = 4'b1010;
    #10;
    D = 4'b1111;
    #10;
    D = 4'b0101;
    #10;
    rst = 1;
    #10;
    rst = 0;
    D = 4'b1100;
    #10;
    $finish;
  end
endmodule
```

Screenshot 1: 4-bit Register – Verilog Design and Testbench Code (EDA Playground)



Screenshot 2: 4-bit Register – Simulation Waveform (EPWave)



Simulation Output

The EPWave simulation verifies the correct operation of the 4-bit Register. Initially, the reset signal clears the register and the output Q = 0000. After reset is de-asserted, the register loads the input data on every positive edge of the clock. The input values 0001, 1010, 1111, 0101, and 1100 are successfully stored in the register. When the reset signal is asserted again, the output returns to 0000. The waveform confirms that the output changes only at the rising edge of the clock, matching the expected behavior of a synchronous 4-bit Register.

Observed Output

Clock Edge	Reset	Input D	Output Q	Observation
Initial	1	XXXX	0000	Register Reset
1st Rising Edge	0	0001	0001	Data Stored
2nd Rising Edge	0	1010	1010	Data Stored
3rd Rising Edge	0	1111	1111	Data Stored
4th Rising Edge	0	0101	0101	Data Stored
Reset Applied	1	XXXX	0000	Register Cleared
Next Rising Edge	0	1100	1100	Data Stored

Waveform Verification

- Reset operation works correctly.
- The register stores the input only on the positive edge of the clock.
- Output remains stable between clock edges.
- The output exactly follows the input after each rising edge.
- The waveform matches the expected behavior of a 4-bit Register.

Result

The 4-bit Register was successfully designed and simulated using Verilog HDL in EDA Playground. The EPWave simulation confirms that the register correctly stores 4-bit input data on each positive clock edge and resets to 0000 when the reset signal is active. Hence, the design and simulation are verified successfully.

Part E: Design and Simulation of a 4-bit Synchronous Up Counter

Aim

To design, implement, and simulate a 4-bit synchronous up counter using Verilog HDL and verify its operation using EDA Playground.

Theory

A 4-bit synchronous up counter is a sequential digital circuit that increments its count by 1 on every positive (rising) edge of the clock signal. Since it uses 4 bits, it can represent 16 different states, ranging from 0000 (0) to 1111 (15). All flip-flops in the counter receive the same clock signal simultaneously, making it a synchronous counter. After reaching the maximum count (1111), the counter automatically wraps around to 0000 and continues counting.

Working

- Initially, the counter output is 0000.
- At every positive edge (posedge) of the clock, the counter increments by 1.

The output changes in the following sequence:

0000 → 0001 → 0010 → 0011 → 0100 → 0101 → 0110 → 0111 → 1000 → 1001 → 1010 → 1011 → 1100 → 1101 → 1110 → 1111 → 0000

This process continues until the simulation ends.

Verilog Design Code (as implemented in EDA Playground)

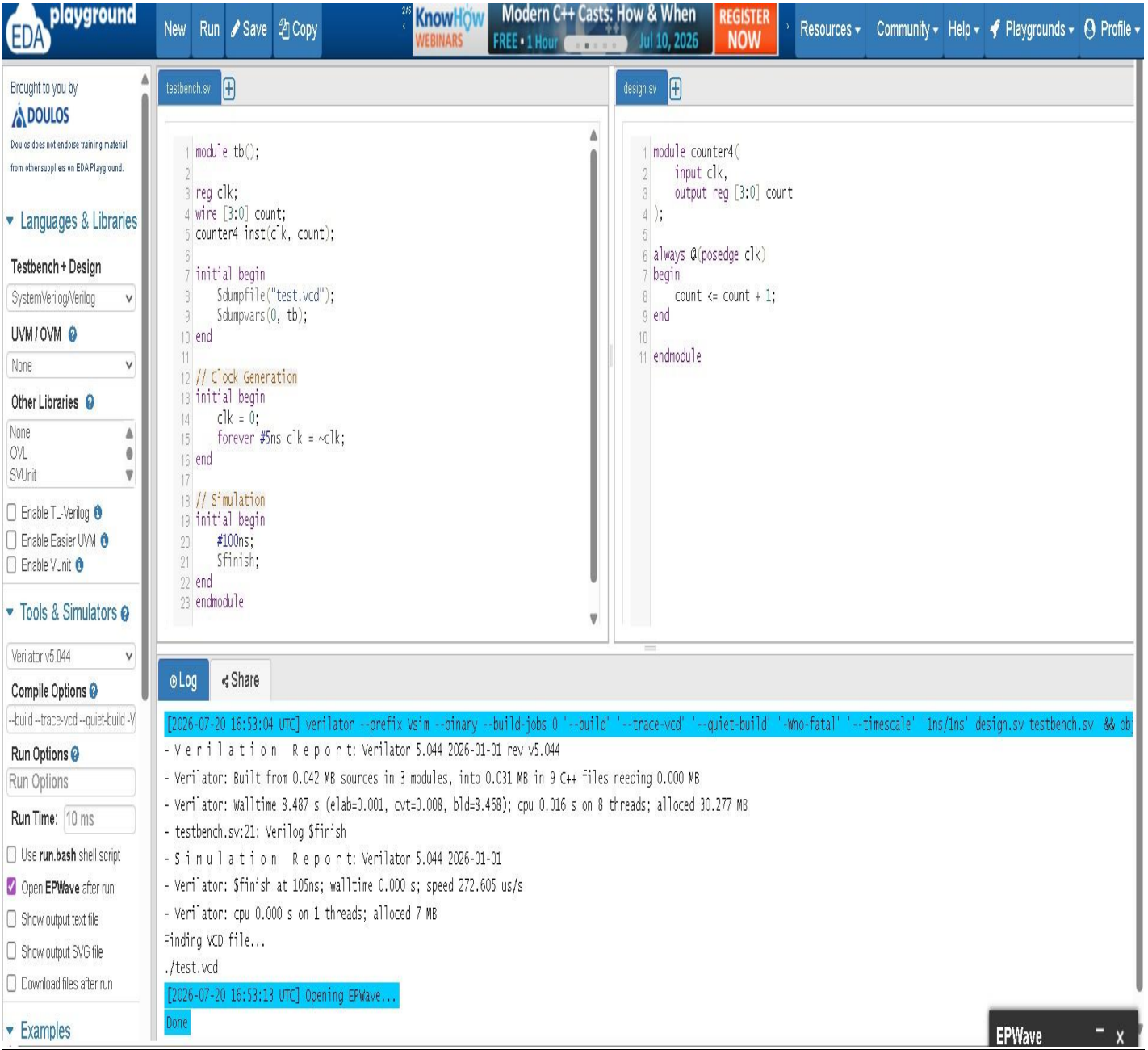
```
module counter4(
    input clk,
    output reg [3:0] count
);
 always @(posedge clk)
  begin
    count <= count + 1;
  end
endmodule
```

Testbench Code

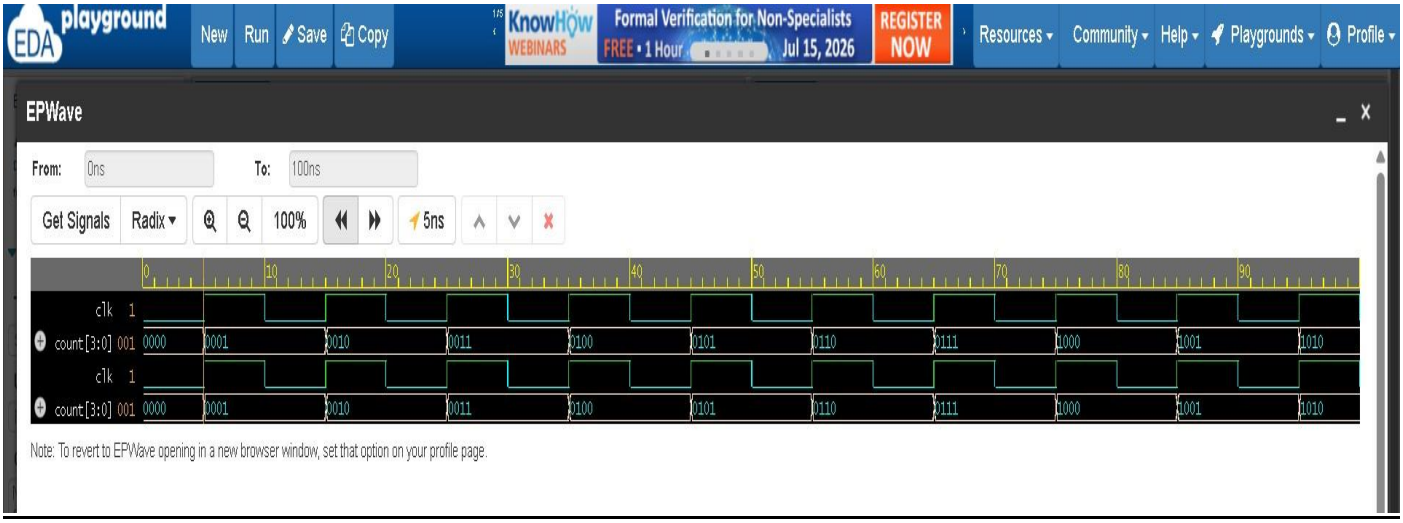
```
module tb();
  reg clk;
  wire [3:0] count;
  counter4 inst(clk, count);
  initial begin
    $dumpfile("test.vcd");
    $dumpvars(0, tb);
  end
  // Clock Generation
  initial begin
    clk = 0;
    forever #5ns clk = ~clk;
  end
```

```
  // Simulation
  initial begin
    #100ns;
    $finish;
  end
endmodule
```

Screenshot 1: 4-bit Counter – Verilog Design and Testbench Code (EDA Playground)



Screenshot 2: 4-bit Counter – Simulation Waveform (EPWave)



Simulation Output

The waveform obtained from EDA Playground shows that the initial counter value is 0000. On the first positive edge of the clock, the counter changes to 0001. On every subsequent positive edge, the counter increments by 1. The output follows the correct binary counting sequence, and no skipped counts, glitches, or unknown (X) values are observed. The waveform confirms that the counter operates correctly as a 4-bit synchronous up counter.

Output Table

Clock (Positive Edge)	Counter Output	Decimal
Initial	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	10

Expected continuation: 1011, 1100, 1101, 1110, 1111, 0000, 0001, ...

Result

The 4-bit synchronous up counter was successfully designed and simulated using Verilog HDL in EDA Playground. The waveform verifies that the counter increments by one on every positive edge of the clock and follows the correct binary counting sequence from 0000 to 1111 before wrapping back to 0000. Hence, the design is verified successfully.

Part F: Simulation and Verification

All modules were compiled and simulated in EDA Playground using Verilator v5.044, with waveforms viewed in EPWave. Each design was driven by a dedicated testbench that generated the clock, applied reset, and exercised all relevant input combinations, and the resulting waveforms were compared against the expected truth tables.

What Was Verified

- Outputs matched the expected truth table / state sequence for every case
- No unknown (X) or floating states were observed
- Correct timing and transition of signals on the waveform
- Synchronous reset behavior verified for the D Flip-Flop, JK Flip-Flop, and 4-bit Register
- Correct free-running increment sequence verified for the 4-bit Counter

Part G: Learning Outcomes

- Understood the difference between combinational and sequential logic
- Learned how clocked always blocks (posedge) are used to model sequential behavior
- Implemented a D Flip-Flop with synchronous reset in Verilog
- Implemented a JK Flip-Flop with No-Change, Reset, Set, and Toggle behavior
- Designed and verified a 4-bit Register with parallel load and reset
- Designed and verified a 4-bit synchronous up counter
- Wrote testbenches to generate clocks and apply test sequences for sequential circuits
- Simulated designs and analyzed waveform outputs using EPWave on EDA Playground
- Verified functional correctness of each circuit against its truth table / expected state sequence

Author
Soha Kalekhan

VLSI Internship Task 3